

Efficient Ensemble for Fine-tuning Language Models on Multiple Datasets

Dongyue Li¹, **Ziniu Zhang**¹, Lu Wang², and Hongyang R. Zhang¹

¹Northeastern University, Boston, MA

²University of Michigan, Ann Arbor, MI



Fine-tuning Language Models on Multiple Datasets

- Many scenarios of fine-tuning language models involve a mix of many datasets. Examples include instruction tuning on a large collection of datasets such as FLAN v2 (Xia et al., 2024) and training a multilingual language model (Neubig and Hu, 2018).
- Directly applying a fine-tuning method like LoRA (Hu et al., 2021) on all tasks will lead to negative interference across tasks.
- Another option is to first pre-train a shared adapter on all the datasets and then fine-tune it on each dataset (Liu et al., 2019). This increases the computational overhead by two.



Fine-tuning Language Models on Multiple Datasets

- Directly applying a fine-tuning method like LoRA (Hu et al., 2021) on all tasks will lead to negative interference across tasks.
- Another option is to first pre-train a shared adapter on all the datasets and then fine-tune it on each dataset (Liu et al., 2019). This increases the computational overhead by two.
- **Case study of fine-tuning Llama-3-8B on SuperGLUE:** We compare the performance between (i) fine-tuning the model on each task and (ii) fine-tuning the model on a pairwise combination of 10 tasks.
 - Using QLoRA and QAdapter (with 4-bit quantization of the base model) increases the negative transfers by **65%** and **75%**.
 - Multitask learning followed by fine-tuning boosts the average accuracy up to **10%**, but uses **2x** more computation cost and **4x** more memory.



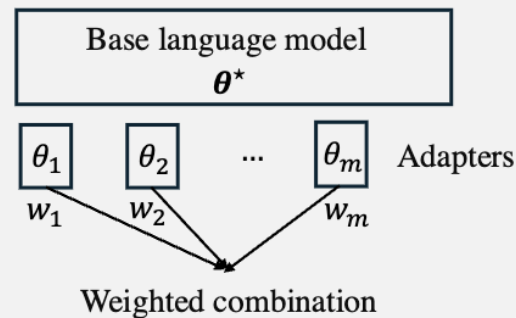
Problem Statement

- Given n datasets and a base adaptation method, we aim to design an ensemble of adapters that applies weighted averaging to their outputs, with minimal computation and memory overhead to the base method.
- First, we partition n datasets into m groups based on estimating the task affinity scores.
- Second, we design an ensemble of adapters by fine-tuning one adapter per group, denoted as $\{\theta_1, \theta_2, \dots, \theta_m\}$, and combining the outputs with weights $\{w_1, w_2, \dots, w_m\}$.

Input: n downstream datasets

$\mathcal{D}_1$  $\mathcal{D}_2$  ... $\mathcal{D}_n$ 

Output: An ensemble of m adapters on a base pretrained language model



Related Works of Measuring Task Similarity

- There are several works about evaluating the similarities between tasks or data samples.
 - **Influence functions** evaluate how slight changes to individual training examples affect model predictions (Koh and Liang, 2017).
 - **Data modeling** uses random sampling and surrogate models to approximate the impact of adding or removing specific training samples on prediction loss (Park et al., 2023; Ilyas et al., 2022).
 - **Task affinity grouping** computes a task affinity matrix by computing gradient similarity between tasks (Fifty et al., 2021).
- Inspired by them, we propose an efficient estimation of model fine-tuning performances on subsets of tasks **without performing any fine-tuning**.



Overview of Our Contributions

- We propose an efficient method for identifying task groups by estimating task affinities without any fine-tuning, using a first-order approximation technique. We evaluate this approach extensively on parameter-efficient fine-tuning methods.
- We designed an ensemble method that uses several low-rank adapters for multiple datasets and validated its performance across diverse multitask scenarios.
- We analyze the generalization of low-rank adapters by measuring their empirical generalization errors and their sharpness.



Task Affinity Grouping for LM Fine-tuning

- **The first step** of our approach is to partition the n given datasets into m disjoint subsets based on their task affinities.
- We estimate affinity scores as follows.
 - Let S be a subset of datasets containing i and j .
 - Let $f_i(S)$ be the fine-tuning performance on dataset i using all datasets in S .
 - The task affinity score of task j to i is:

$$T_{i,j} = \frac{1}{n_{i,j}} \sum_{S_k} f_i(S_k)$$

where $\{S_k\}$ are subsets sampled from $\{1, \dots, n\}$ and $n_{i,j}$ is the number of random subsets that contain both i, j .

- Computing $T_{i,j}$ requires fine-tuning the model on multiple subset combinations. How can we design an efficient algorithm to quickly estimate $f_i(S)$?



A Gradient-based Estimation Algorithm

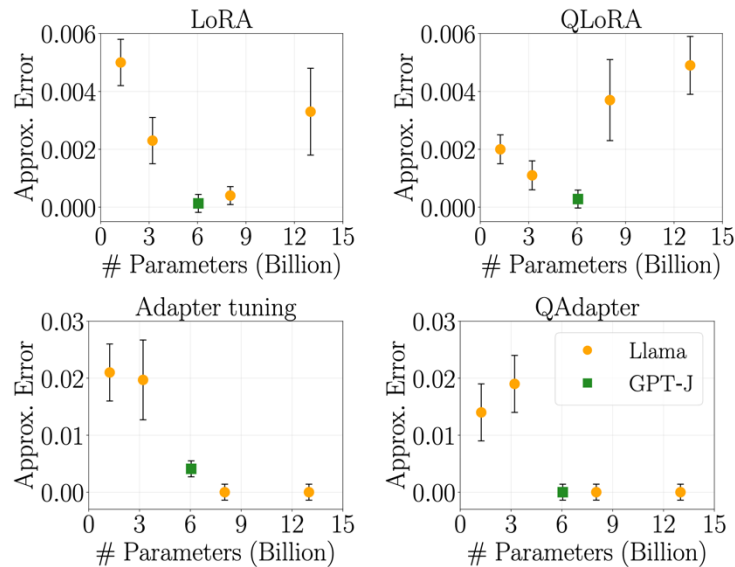
- Fine-tuned adapters stay close to the base model.
 - On 50 random subsets of tasks, the relative distance of the fine-tuned adapter to the pretrained model is less than **0.2%** on average.

	Llama-3-1B	Llama-3-3B	Llama-3-8B
LoRA	$0.16_{\pm 0.04}\%$	$0.14_{\pm 0.02}\%$	$0.12_{\pm 0.02}\%$
QLoRA	$0.18_{\pm 0.02}\%$	$0.16_{\pm 0.03}\%$	$0.11_{\pm 0.02}\%$
Adapter	$0.09_{\pm 0.03}\%$	$0.05_{\pm 0.01}\%$	$0.08_{\pm 0.02}\%$
QAdapter	$0.11_{\pm 0.03}\%$	$0.08_{\pm 0.03}\%$	$0.07_{\pm 0.01}\%$



A Gradient-based Estimation Algorithm

- Fine-tuned adapters stay close to the base model.
 - On 50 random subsets of tasks, the relative distance of the fine-tuned adapter to the pretrained model is less than **0.2%** on average.
- When the fine-tuned adapter parameter X is close to the initialization θ^* , the model output $h_X(s, y)$ can be approximated by a first-order Taylor's expansion centered at the meta-initialization θ^* .



$$h_X(s, y) \approx h_{\theta^*}(s, y) + \nabla_X h_{\theta^*}(s, y)^\top (X - \theta^*) + \epsilon$$

- We empirically measure ϵ across LMs from 1B to 34B over 50 random task subsets.
- The Relative RSS is less than **3%** on average, across LoRA, adapter, QLoRA, and Qadapter.



A Gradient-based Estimation Algorithm

- **Estimating fine-tuned adapter parameters:**

- Apply the first-order approximation to the log-loss of the model:

$$\ell(X) = \log(1 + \exp(-y_i h_X(s_i, y_i)))$$

$$\Rightarrow \hat{\ell}(X) = \log(1 + \exp(-y_i g_i^\top (X - \theta^*) - y_i h_{\theta^*}(s_i, y_i))).$$

- For S with training samples denoted as D_S , we estimate fine-tuned adapter weights by minimizing averaged $\hat{\ell}(X)$ on D_S .

$$\widehat{\theta}_S \leftarrow \arg \min_{x \in \mathbb{R}^p} \frac{1}{n_S} \sum_{(s,y) \in D_S} \hat{\ell}(X)$$

- This corresponds to solving logistic regression on a subset of tasks S , using their gradient $g_i = \nabla_X h_{\theta^*}(s_i, y_i)$ as the feature vector and y_i as the response label.
- Solving logistic regression on CPUs takes within **2 seconds per instance!**



A Gradient-based Estimation Algorithm

- **Estimating fine-tuned adapter parameters:**

- Apply the first-order approximation to the log-loss of the model:

$$\ell(X) = \log(1 + \exp(-y_i h_X(s_i, y_i)))$$

$$\Rightarrow \hat{\ell}(X) = \log(1 + \exp(-y_i g_i^\top (X - \theta^*) - y_i h_{\theta^*}(s_i, y_i))).$$

- For S with training samples denoted as D_S , we estimate fine-tuned adapter weights by minimizing averaged $\hat{\ell}(X)$ on D_S .

$$\hat{\theta}_S \leftarrow \arg \min_{x \in \mathbb{R}^p} \frac{1}{n_S} \sum_{(s,y) \in D_S} \hat{\ell}(X)$$

- This corresponds to solving logistic regression on a subset of tasks S , using their gradient $g_i = \nabla_X h_{\theta^*}(s_i, y_i)$ as the feature vector and y_i as the response label.
- We apply this estimation to compute task affinity scores for every pair of tasks. Given a $n \times n$ task affinity matrix, we apply a clustering algorithm to partition n datasets into m groups.



Designing an ensemble of adapters

- After task grouping, we fine-tune one adapter on tasks in each group, yielding m adapters.
- Then, we apply b additional gradient boosting steps to reduce the loss of groups with high training losses.
 - At step i , we choose a group with the highest training error.
 - We fit a new adapter to predict $1 - p_s$ where p_s is the correct class prediction probability on the sample.
 - We approximate the above loss based on the gradient estimation, leading to the following linear regression on the selected group.
- The final ensemble contains a total of $M = m + b$ adapters.



Comparison to Existing Methods

- In summary, our method uses the runtime of $T + G$, where T is the runtime of the base fine-tuning method and G is the cost of evaluating the gradients once for all the samples.
- The memory cost of our method is MA , where A is the memory used by one adapter, including activation memory.

Approach	Runtime	Memory
Base fine-tuning	T	A
Pre-train then fine-tune	$\approx 2T$	nA
Our ensemble method	$T + G$	MA

- Compared to the base fine-tuning method, the additional cost of G is small compared to the fine-tuning cost of T , less than 10%.
- Compared to pretraining followed by fine-tuning, our method saves approximately half the computational cost and $\frac{n}{M}$ memory.



Results for Approximating Fine-tuning Losses

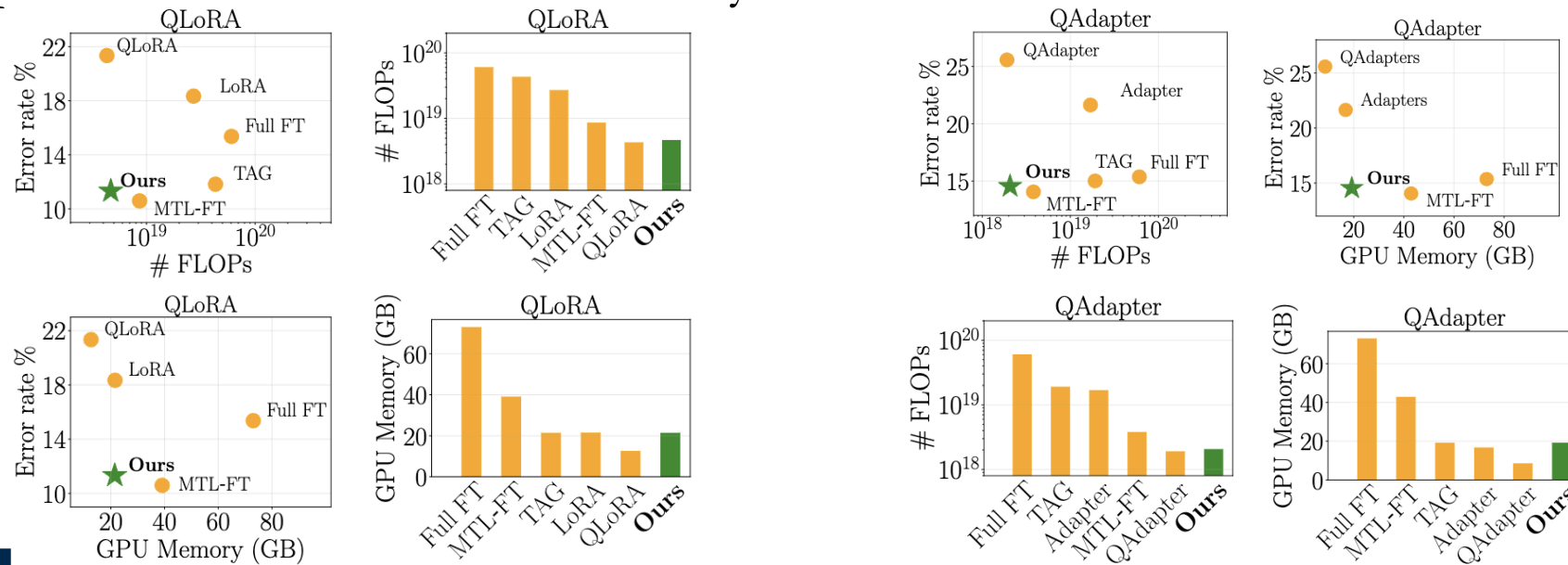
- We evaluate the error between estimated and true fine-tuning performances on 50 random task subsets.
- We apply a dimension projection based on the JL lemma to further reduce the computation. We vary the projected dimension from 200 to 800.
- Our approach achieves a relative error within 9% while using 105x less computation than actual fine-tuning.

d	Llama-3-1B	Llama-3-3B	Llama-3-8B	Speedup
200	8.2%	8.1%	7.0%	105×
400	4.7%	4.8%	4.3%	105×
800	4.6%	4.4%	4.2%	105×



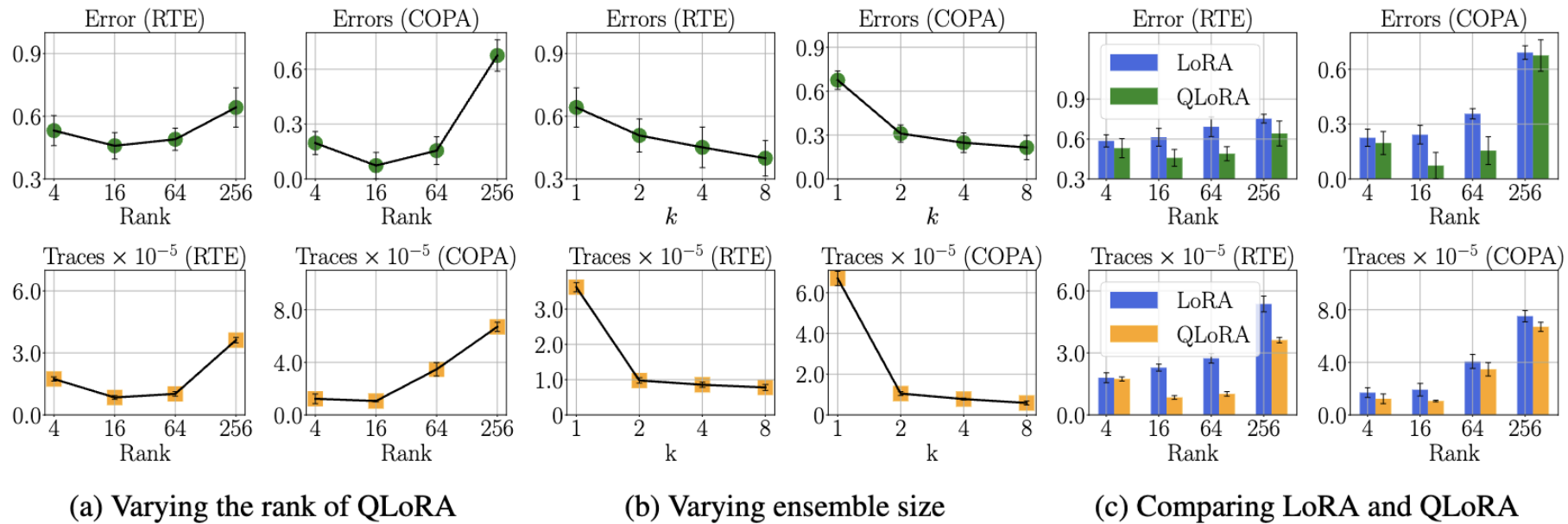
Trade-off of the Test Accuracy and Efficiency

- Compared to QLoRA, we improve accuracy by **10%** with only 8% additional computation and 9 GB more memory. The result on QAdapter is similar.
- Compared to pretraining followed by fine-tuning and task grouping, we achieve comparable performance while reducing computation and memory by **45%**
- Compared to full fine-tuning, we improve accuracy by **4%** with only 92% less computation and 52 GB less memory.



Generalization of Low-Rank Adaptation

- We evaluate the empirical generalization errors and sharpness by the trace of Hessian.
 - Relatively small adapters often achieve the best generalization performance.
 - Ensembles of adapters yield lower generalization errors than individual adapters.
 - Adapters trained on quantized models exhibit smaller generalization errors compared to those trained on full-precision models.



Summary

- We present an ensemble method of low-rank adapters for adapting language models across multiple datasets.
 - First, we develop an efficient task affinity grouping algorithm, including a first-order approximation for estimating task affinities and a clustering step.
 - Secondly, we construct an ensemble for the groups. It consistently improves fine-tuning performance with minimal computational overhead.
 - Lastly, we analyze the sharpness measures of low-rank adapters.

Thanks for Listening!

- Paper: <https://arxiv.org/pdf/2505.21930>
- Code: <https://github.com/VirtuosoResearch/EnsembleLoRA>.
- Emails: {li.dongyu, zhang.zini, ho.zhang}@northeastern.edu, wangluxy@umich.edu

